

Vérification Formelle d'Applications Critiques avec Réseaux de Petri

Sujet 4 : Système de triage médical

Auteurs :

Bey Benjamin, FRANÇOIS Noé, El
Haj-Benali Anas, ARES-WAGNER
Baptiste, CHURCH William, Ouhab
Mohammed

Année universitaire :
2025 - 2026

Résumé. Ce rapport présente la modélisation et la vérification formelle complète d'un système critique de triage médical par réseaux de Petri colorés (CPN). Après un état de l'art comparé des principales méthodes formelles utilisées dans les systèmes critiques, le système est instancié en un CPN à neuf places, cinq transitions et une garde de priorité. L'analyse structurelle — matrice d'incidence et quatre P-invariants — complète l'exploration exhaustive de l'espace des marquages réalisée par un model checker implémenté en Python. Quatre propriétés critiques (sûreté, vivacité, absence d'interblocage, respect des priorités) sont formalisées en LTL et CTL, prouvées logiquement et vérifiées automatiquement sur l'intégralité des 7 marquages atteignables.

Mots-clés : réseaux de Petri colorés, méthodes formelles, model checking, LTL, CTL, P-invariants, triage médical.

Contents

1	Introduction	3
2	État de l'art des méthodes formelles	3
2.1	Logiques temporelles	3
2.2	Automates	3
2.3	Réseaux de Petri	3
2.4	Algèbres de processus et autres approches	4
2.5	Outils de model checking	4
2.6	Types de propriétés vérifiées	4
2.7	Comparaison avantages / limites par domaine	4
2.8	Pourquoi les réseaux de Petri colorés pour ce projet ?	5
3	Description du système choisi	5
3.1	Contexte et fonctionnement global	5
3.2	Éléments critiques et risques associés	5
3.3	Conformité aux standards	5
4	Modélisation par réseau de Petri coloré	6
4.1	Représentation graphique	6
4.2	Définition formelle	6
4.3	Instanciation pour le système de triage	6
4.4	Analyse structurelle : matrice d'incidence et P-invariants	7
4.5	Règles de franchissement	8
4.6	Justification des choix de modélisation	8
5	Expression formelle des propriétés	8
5.1	Propriété 1 — Sûreté	8
5.2	Propriété 2 — Vivacité	9
5.3	Propriété 3 — Absence d'interblocage	9
5.4	Propriété 4 — Respect des priorités	9
5.5	Synthèse	9

6	Implémentation et model checking	9
6.1	Simulateur du réseau de Petri	9
6.2	Algorithme de model checking exhaustif	11
6.3	Complexité et limites	13
7	Résultats du model checking	14
7.1	Scénario retenu	14
7.2	Trace du simulateur	14
7.3	Graphe des marquages atteignables	14
7.4	Vérification de toutes les propriétés et P-invariants	15
7.5	Discussion des résultats	15
8	Conclusion et perspectives	15
8.1	Bilan	15
8.2	Limites et extensions possibles	16
8.3	Apprentissages	16

1 Introduction

Les systèmes critiques — dont une défaillance peut entraîner des pertes humaines ou des catastrophes — sont présents dans l’aéronautique (DO-178C), le ferroviaire (EN 50128), le spatial et le médical. Dans ce dernier domaine, la norme **IEC 62304** [12] exige, pour les logiciels de niveau C (défaillance pouvant causer des blessures graves), l’application de méthodes formelles.

Le test empirique classique est insuffisant : il ne couvre qu’un nombre fini de scénarios choisis sans garantir l’exhaustivité. Les méthodes formelles apportent une réponse rigoureuse reposant sur trois piliers : (1) un modèle mathématique précis, (2) une spécification formelle des propriétés attendues, (3) une vérification automatique et exhaustive.

Ce rapport applique le *model checking* à un système de triage médical modélisé par un CPN : représentatif des systèmes à ressources partagées où la correction du protocole de priorité est critique pour la sécurité du patient. L’organisation est : état de l’art (section 2), description du système (section 3), modélisation (section 4), propriétés formelles (section 5), implémentation (section 6), résultats (section 7), conclusion (section 8).

2 État de l’art des méthodes formelles

2.1 Logiques temporelles

Les logiques temporelles permettent d’exprimer des propriétés portant sur l’évolution d’un système.

LTL (Linear Temporal Logic). Introduite par Pnueli (1977) [3], la LTL raisonne sur des exécutions linéaires. Opérateurs : $\mathbf{X} \varphi$ (*next*), $\mathbf{F} \varphi$ (*eventually*), $\mathbf{G} \varphi$ (*globally*), $\varphi \mathbf{U} \psi$ (*until*).

CTL (Computation Tree Logic). Développée par Clarke–Emerson [4] et Queille–Sifakis [5], la CTL raisonne sur l’arbre des exécutions. Quantificateurs : A (tous les chemins), E (un chemin). Combinaisons fréquentes : **AG**, **AF**, **EX**.

CTL* et μ -calcul. CTL* englobe LTL et CTL. Le μ -calcul modal est le plus expressif et sert souvent de langage cible interne aux outils.

2.2 Automates

Un **automate fini** (états, transitions) est adapté aux protocoles mais explose combinatoirement avec la concurrence. Les **automates temporisés** (Alur & Dill, 1994 [6]) ajoutent des horloges réelles pour les systèmes temps-réel. L’outil de référence est UPPAAL.

2.3 Réseaux de Petri

Les réseaux de Petri (RdP), introduits par Petri (1962) [7], sont des graphes bipartis (places, transitions, arcs, jetons) qui modélisent naturellement la concurrence et le partage de ressources. Les **réseaux de Petri colorés** (CPN, Jensen 1992 [8]) attachent une couleur (type) à chaque jeton, combinant flux de contrôle et flux de données.

2.4 Algèbres de processus et autres approches

CSP (Hoare [9]) et CCS (Milner [10]) décrivent un système comme une composition de processus communicants. D'autres approches complètent le paysage : démonstrateurs de théorèmes (Coq, Isabelle/HOL), raffinement formel (Event-B, utilisé pour la ligne 14 du métro parisien), interprétation abstraite, et solveurs SAT/SMT.

2.5 Outils de model checking

Table 1: Principaux outils de model checking.

Outil	Formalisme / Logique	Spécialité
SPIN	Promela ; LTL	Protocoles concurrents (NASA)
NuSMV	FSM ; CTL et LTL	Model checking symbolique (BDD) et borné (SAT)
UPPAAL	Automates temporisés ; TCTL	Systèmes temps-réel, avionique
LoLA	Réseaux de Petri ; CTL	Vérification très efficace de grands RdP
TINA	RdP et RdP temporels ; LTL/CTL	Graphes d'états (LAAS-CNRS)
CPN Tools	CPN ; propriétés CPN	Édition, simulation, analyse des CPN

2.6 Types de propriétés vérifiées

- **Sûreté** (*safety*) : $\mathbf{G} \neg \text{mauvais}$ — violation par un préfixe fini.
- **Vivacité** (*liveness*) : $\mathbf{F} \text{bon}$.
- **Absence de deadlock** : aucun état non-final ne bloque.
- **Invariants** : propriété vraie dans tout état atteignable ; calculables structurellement (P-invariants).
- **Bornitude** : le nombre de jetons reste borné, assurant un espace d'états fini.

2.7 Comparaison avantages / limites par domaine

Table 2: Comparaison des méthodes formelles selon le type d'application critique.

Méthode	Conc.	Données	T.-réel	Échelle	Domaines typiques
LTL + SPIN	✓	✗	✗	Moyen	Protocoles, embarqué
CTL + NuSMV	✓	~	✗	Bon (BDD)	Matériel, pilotes
UPPAAL	✓	✗	✓	Bon	Avionique, médical, freinage
RdP + LoLA	✓	✗	✗	Très bon	Manufactures, logistique
CPN + CPN Tools	✓	✓	✗	Moyen	Protocoles, distribué
CSP + FDR	✓	✗	✗	Bon	Protocoles, ferroviaire (UK)
Event-B/Rodin	✓	✓	✗	Bon	Métro automatique, ferroviaire

Analyse. LTL et CTL sont complémentaires : LTL exprime mieux les propriétés de trace linéaire, CTL les propriétés de branchement (existence d'un chemin sûr). UPPAAL est le choix privilégié dès que des contraintes temporelles quantitatives entrent

en jeu (dispositifs médicaux, avionique). Les CPN se distinguent pour les systèmes à ressources partagées avec données typées.

2.8 Pourquoi les réseaux de Petri colorés pour ce projet ?

Le système de triage présente trois caractéristiques clés : partage de ressources, politique de priorité sur les données (couleur des jetons) et absence de contraintes temporelles quantitatives strictes. Les CPN offrent le meilleur compromis : modèle compact, logique de priorité dans la garde, et riche panoplie d’analyses structurelles (P-invariants, bornitude).

3 Description du système choisi

3.1 Contexte et fonctionnement global

Le système modélisé est un poste de triage médical d’un service d’urgences hospitalier dans sa version simplifiée : un unique médecin, un unique siège de consultation, une salle d’attente. Chaque patient reçoit à son entrée l’un des trois niveaux d’urgence :

- **Urgence absolue** : pronostic vital engagé, priorité maximale ;
- **Urgent** : prioritaire sur les patients non urgents ;
- **Non urgent** : pris en charge lorsque médecin et siège sont libres et qu’aucun patient plus prioritaire n’attend.

Trois événements élémentaires : (1) entrée d’un patient, (2) appel du patient le plus prioritaire (médecin libre + siège libre), (3) fin de consultation et libération des ressources.

3.2 Éléments critiques et risques associés

- **Sécurité** : un médecin ne peut prendre en charge deux patients simultanément (règle déontologique et physique).
- **Vivacité** : aucun patient ne doit rester bloqué indéfiniment (risque de famine).
- **Priorité** : un patient en urgence absolue ne doit jamais attendre qu’un patient non urgent soit traité en premier (inversion de priorité).

3.3 Conformité aux standards

La norme **IEC 62304** [12] classe les logiciels médicaux en trois niveaux (A, B, C). Pour le niveau C (défaillance pouvant causer la mort), des activités formelles de vérification sont explicitement requises. Notre système — où une inversion de priorité peut engager le pronostic vital — relève de ce niveau. L’approche de ce projet est donc directement en adhérence avec ces exigences.

4 Modélisation par réseau de Petri coloré

4.1 Représentation graphique

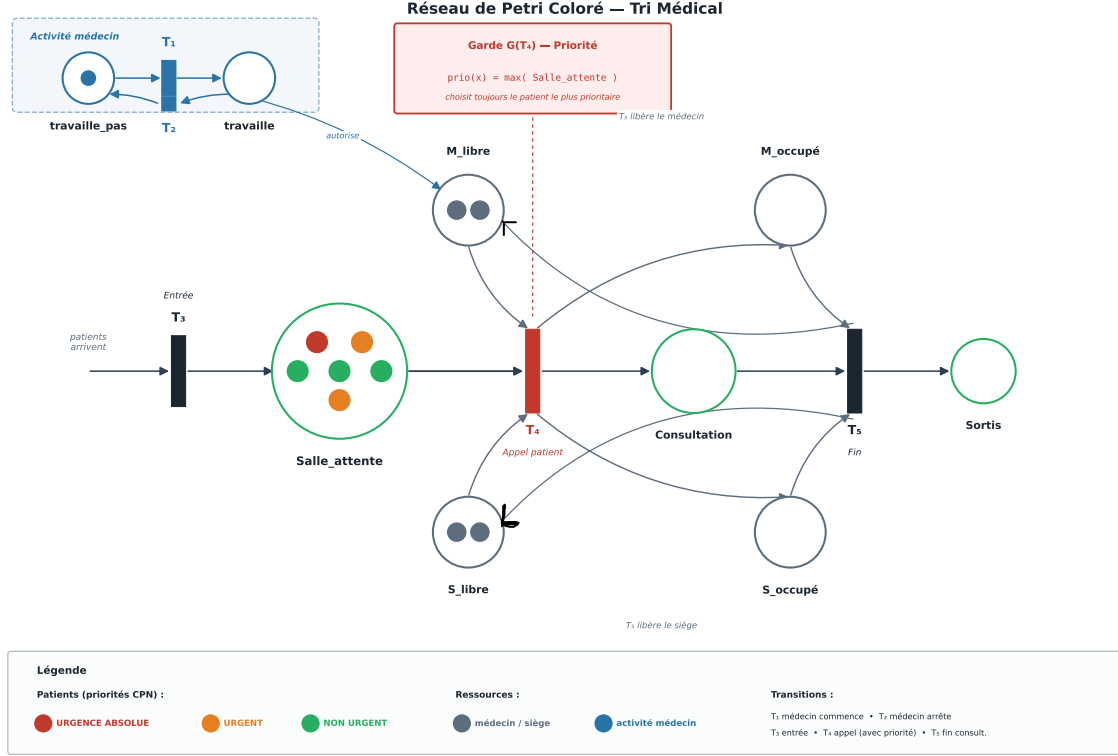


Figure 1: Réseau de Petri coloré du système de triage médical (sens des flèches indiqués en noir). Jetons colorés : rouge (urgence absolue), orange (urgent), vert (non urgent). Jetons gris : ressources (médecin, siège). Le sous-réseau *Activité médecin* (P_0/P_1 , T_1/T_2) modélise les phases de service du médecin ; l'arc *autorise* conditionne T_4 à la présence d'un jeton dans P_1 . T_3 : entrée patient ; T_4 : appel prioritaire ; T_5 : fin de consultation et restitution des ressources.

4.2 Définition formelle

Définition 4.1 (Réseau de Petri coloré). *Un réseau de Petri coloré est un n -uplet $\mathcal{N} = (P, T, F, C, G, W, M_0)$ où : P est un ensemble fini de places ; T un ensemble fini de transitions avec $P \cap T = \emptyset$; $F \subseteq (P \times T) \cup (T \times P)$ la relation de flux ; C l'ensemble des couleurs ; $G : T \rightarrow \mathbf{Bool}$ la garde ; W la fonction de poids des arcs ; M_0 le marquage initial.*

4.3 Instanciation pour le système de triage

Couleurs. $C = \{\text{URGENCE_ABSOLUE}, \text{URGENT}, \text{NON_URGENT}, \text{ressource}\}$

Fonction de priorité. $prio(\text{URGENCE_ABSOLUE}) = 3$, $prio(\text{URGENT}) = 2$, $prio(\text{NON_URGENT}) = 1$.

Table 3: Places du réseau et leur marquage initial.

Place	Signification	Marquage initial
P_0 : travaille_pas	Médecin hors service	1 jeton ressource
P_1 : travaille	Médecin en service	$\emptyset$
P_2 : Salle_attente	Patients en attente	multiensemble de patients
P_3 : M_libre	Médecin disponible	1 jeton ressource
P_4 : M_occupé	Médecin en consultation	$\emptyset$
P_5 : S_libre	Siège disponible	1 jeton ressource
P_6 : S_occupé	Siège en consultation	$\emptyset$
P_7 : Consultation	Patient en cours de traitement	$\emptyset$
P_8 : Sortis	Patients traités et sortis	$\emptyset$

Table 4: Transitions du réseau de Petri.

Transition	Rôle	Garde / préconditions
T_1 : Médecin commence	Prise de service : $P_0 \rightarrow P_1$	vrai
T_2 : Médecin arrête	Fin de service : $P_1 \rightarrow P_0$	vrai
T_3 : Entrée	Nouveau patient dans P_2	vrai
T_4 : Appel patient	$P_2, P_3, P_5, P_1 \rightarrow P_4, P_6, P_7$	$G(T_4)$: prio(x) = max(P_2) ; arc autorise : $P_1 \geq 1$
T_5 : Fin consultation	$P_7, P_4, P_6 \rightarrow P_8, P_3, P_5$	vrai

4.4 Analyse structurelle : matrice d'incidence et P-invariants

Matrice d'incidence. La matrice $C \in \mathbb{Z}^{9 \times 5}$ encode les variations de marquage lors du franchissement de chaque transition :

Table 5: Matrice d'incidence C du réseau (lignes : places, colonnes : transitions).

Place	T_1	T_2	T_3	T_4	T_5
P_0 : travaille_pas	-1	+1	0	0	0
P_1 : travaille	+1	-1	0	0	0
P_2 : Salle_attente	0	0	+1	-1	0
P_3 : M_libre	0	0	0	-1	+1
P_4 : M_occupé	0	0	0	+1	-1
P_5 : S_libre	0	0	0	-1	+1
P_6 : S_occupé	0	0	0	+1	-1
P_7 : Consultation	0	0	0	+1	-1
P_8 : Sortis	0	0	0	0	+1

P-invariants. Un P-invariant (P-semiflow) est un vecteur $\mathbf{y} \geq \mathbf{0}$ tel que $\mathbf{y}^\top C = \mathbf{0}$; il définit une quantité conservée dans tout état atteignable. Le calcul par réduction gaussienne conduit aux quatre P-invariants indépendants :

P-invariant	Équation	Interprétation
y_1	$P_0 + P_1 = 1$	Sous-réseau d'activité cohérent
y_2	$P_3 + P_4 = 1$	Un seul médecin (libre ou occupé)
y_3	$P_5 + P_6 = 1$	Un seul siège (libre ou occupé)
y_4	$P_2 + P_7 + P_8 = n$	Conservation des patients (n fixé)

y_2 garantit structurellement la sûreté : $P_3 + P_4 = 1$ implique $P_4 \leq 1$, i.e. au plus un patient peut être en consultation à tout moment. y_4 est valide dans le modèle fermé (T_3 non franchissable, patients pré-chargés).

4.5 Règles de franchissement

T_4 est franchissable si et seulement si : (1) $P_3 \geq 1$, (2) $P_5 \geq 1$, (3) $P_2 \geq 1$, (4) $P_1 \geq 1$ (arc *autorise* : médecin en service), (5) $G(T_4)$: le patient x choisi vérifie $\text{prio}(x) = \max\{\text{prio}(y) \mid y \in P_2\}$. Son franchissement retire x de P_2 , transfère $P_3 \rightarrow P_4$, $P_5 \rightarrow P_6$, et dépose x dans P_7 .

T_5 est franchissable dès que $P_7 \geq 1$. Il déplace le patient de P_7 vers P_8 et restitue $P_4 \rightarrow P_3$ et $P_6 \rightarrow P_5$.

4.6 Justification des choix de modélisation

- **Jetons colorés** : une seule place P_2 pour les trois niveaux d'urgence ; la priorité est capturée par $G(T_4)$ sans duplication.
- **Couples libre/occupé** : $P_3 + P_4 = 1$ (P-invariant y_2) garantit structurellement la sûreté.
- **Place Sortis explicite** : facilite la vérification de vivacité finale ($|P_8| = n$ en fin d'exécution).
- **Sous-réseau d'activité** : T_1/T_2 modélisent les phases de service (pauses, changements de garde) ; abstrait dans l'implémentation pour maintenir un espace d'états fini.

5 Expression formelle des propriétés

5.1 Propriété 1 — Sûreté

Objectif : un médecin ne peut pas prendre deux patients simultanément.

p : médecin libre ; q : médecin occupé ; r : deuxième patient en consultation. $KB_1 = \{q \Rightarrow \neg p, r \Rightarrow p\}$

Proof. Supposons par l'absurde $q \wedge r$. De $q \Rightarrow \neg p$ on tire $\neg p$; de $r \Rightarrow p$ on tire p . Contradiction : $p \wedge \neg p$. *Note structurelle* : le P-invariant y_2 ($P_3 + P_4 = 1$) garantit $P_4 \leq 1$, ce qui prouve la propriété pour toute configuration. $\square$

LTL : $\mathbf{G} \neg(q \wedge r)$ CTL : $\mathbf{AG} \neg(q \wedge r)$

5.2 Propriété 2 — Vivacité

Objectif : tout patient qui arrive finit par être pris en charge.

p : patient en P_2 ; q : médecin libre ; r : patient pris en charge.

$$KB_2 = \{p \Rightarrow \mathbf{F} q, \quad p \wedge q \Rightarrow r, \quad \mathbf{G}(p \Rightarrow (p \mathbf{U} r))\}$$

Le troisième axiome exprime que les patients ne quittent pas P_2 spontanément.

Proof. Soit p vrai. Par $p \Rightarrow \mathbf{F} q$, il existe t^* où q est vrai. Par $\mathbf{G}(p \Rightarrow (p \mathbf{U} r))$, p reste vrai jusqu'à r ; comme r n'est pas encore atteint en t^* , on a p vrai en t^* . De $p \wedge q \Rightarrow r$ par modus ponens, on conclut r en t^* . Donc $\mathbf{F} r$. $\square$

$$\text{LTL} : \mathbf{G}(p \Rightarrow \mathbf{F} r) \quad \text{CTL} : \mathbf{AG}(p \Rightarrow \mathbf{AF} r)$$

5.3 Propriété 3 — Absence d'interblocage

Objectif : le système ne peut jamais se figer en présence de patients.

Soit att : « $P_2 > 0$ ou $P_7 > 0$ » et $deadlock$: « aucune transition franchissable ».

Proof. (a) Si $P_2 > 0$ et $P_3 = P_5 = 1$: T_4 est franchissable. (b) Si $P_7 > 0$: T_5 est franchissable. (c) Si $P_2 > 0$, $P_3 = 0$ ou $P_5 = 0$: alors $P_4 = 1$ ou $P_6 = 1$, donc $P_7 > 0$ (patient en consultation) et T_5 est franchissable. Dans tous les cas où att est vrai, au moins une transition est franchissable. $\square$

$$\text{LTL} : \mathbf{G}(att \Rightarrow \neg deadlock) \quad \text{CTL} : \mathbf{AG}(att \Rightarrow \mathbf{EX} \top)$$

5.4 Propriété 4 — Respect des priorités

Objectif : un patient plus prioritaire en attente est toujours appelé avant un patient moins prioritaire.

Soit $appel(c)$: patient de couleur c entré dans P_7 ; $att(c')$: patient de couleur c' présent dans P_2 .

Proof. La garde $G(T_4)$ impose $prio(x) = \max_{y \in P_2} prio(y)$. Si $prio(c') > prio(c)$ et $att(c')$, alors T_4 ne peut pas choisir x de couleur c : $appel(c)$ est impossible. $\square$

$$\text{LTL} : \mathbf{G}(appel(c) \Rightarrow \neg att(c'), c' > c) \quad \text{CTL} : \mathbf{AG}(\dots)$$

5.5 Synthèse

Table 6: Récapitulatif des propriétés en LTL et CTL.

Propriété	LTL	CTL
Sûreté	$\mathbf{G} \neg(q \wedge r)$	$\mathbf{AG} \neg(q \wedge r)$
Vivacité	$\mathbf{G}(p \Rightarrow \mathbf{F} r)$	$\mathbf{AG}(p \Rightarrow \mathbf{AF} r)$
Absence de deadlock	$\mathbf{G}(att \Rightarrow \neg deadlock)$	$\mathbf{AG}(att \Rightarrow \mathbf{EX} \top)$
Respect des priorités	$\mathbf{G}(appel(c) \Rightarrow \neg att(c'), c' > c)$	$\mathbf{AG}(\dots)$

6 Implémentation et model checking

6.1 Simulateur du réseau de Petri

```

1 PRIORITES = {"NON_URGENT": 1, "URGENT": 2, "URGENCE_ABSOLUE": 3}
2
3 places = {
4     "medecin_libre": 1, "medecin_occupe": 0,
5     "siege_libre": 1, "siege_occupe": 0,
6     "salle_attente": [], "consultation": [], "sortis": []
7 }
8 ordre_attendu = []
9
10 def verifier_securite():
11     # Propriété 1 : jamais deux patients en consultation
12     return (len(places["consultation"]) <= 1
13             and places["medecin_occupe"] <= 1)
14
15 def verifier_ressources():
16     # P-invariants I2 et I3 : P3+P4=1 et P5+P6=1
17     return (places["medecin_libre"] + places["medecin_occupe"] == 1
18             and places["siege_libre"] + places["siege_occupe"] ==
19             1)
20
21 def verifier_priorite_sim():
22     # Propriété 4 : aucun patient plus prioritaire en attente
23     if not places["consultation"]:
24         return True
25     prio = PRIORITES[places["consultation"][0]["priorite"]]
26     return all(PRIORITES[p["priorite"]] <= prio
27               for p in places["salle_attente"])
28
29 def verifier_absence_deadlock():
30     # Retourne True si AUCUN interblocage
31     if (places["salle_attente"] and places["medecin_libre"] == 1
32         and places["siege_libre"] == 1):
33         return True # T4 franchissable
34     if places["consultation"]:
35         return True # T5 franchissable
36     if not places["salle_attente"] and not places["consultation"]:
37         return True # état terminal légitime
38     return False # interblocage réel
39
40 def verifier_vivacite_finale():
41     return (len(places["salle_attente"]) == 0
42             and len(places["consultation"]) == 0)

```

Listing 1: Structures de données et fonctions de vérification.

```

1 def entree_patient(nom, priorite):
2     patient = {"nom": nom, "priorite": priorite}
3     places["salle_attente"].append(patient)
4     ordre_attendu.append(patient)
5     ordre_attendu.sort(key=lambda p: PRIORITES[p["priorite"]],
6                       reverse=True)
7     print(f"\n{n{nom}} arrive (priorité {priorite}).")
8
9 def appel_patient():
10     if (places["medecin_libre"] == 1 and places["siege_libre"] == 1
11         and places["salle_attente"]):
12         # Garde G(T4) : sélection du patient le plus prioritaire

```

```

12     patient = max(places["salle_attente"],
13                   key=lambda p: PRIORITES[p["priorite"]])
14     places["salle_attente"].remove(patient)
15     places["medecin_libre"] -= 1; places["medecin_occupe"] +=
16     1
17     places["siege_libre"] -= 1; places["siege_occupe"] +=
18     1
19     places["consultation"].append(patient)
20     print(f"\n{patient['nom']} appelé en consultation.")
21
22 def fin_consultation():
23     if places["consultation"]:
24         patient = places["consultation"].pop(0)
25         places["sortis"].append(patient)
26         places["medecin_occupe"] -= 1; places["medecin_libre"] +=
27         1
28         places["siege_occupe"] -= 1; places["siege_libre"] +=
29         1
30         print(f"\n{patient['nom']} termine et sort.")
31
32 def main():
33     print("Début de la simulation du réseau de Petri médical")
34
35     entree_patient("A", "NON_URGENT")
36     entree_patient("B", "URGENCE_ABSOLUE")
37     entree_patient("C", "URGENT")
38
39     while places["salle_attente"] or places["consultation"]:
40         if places["salle_attente"]:
41             appel_patient()
42         if places["consultation"]:
43             fin_consultation()
44
45     if verifier_vivacite_finale():
46         print("\nVivacité respectée : tous les patients ont été
47         pris en charge.")
48
49 if __name__ == "__main__":
50     main()

```

Listing 2: Implémentation des transitions.

6.2 Algorithme de model checking exhaustif

L'algorithme explore l'espace des marquages par BFS et vérifie les **quatre propriétés** et les **trois P-invariants** sur chaque marquage atteignable.

```

1 from copy import deepcopy
2 from collections import deque
3
4 PRIORITES = {"NON_URGENT": 1, "URGENT": 2, "URGENCE_ABSOLUE": 3}
5
6 def rendre_hashable(e):
7     return (e["medecin_libre"], e["medecin_occupe"],
8           e["siege_libre"], e["siege_occupe"],
9           e["salle_attente"], e["consultation"], e["sortis"])

```

```

10
11 def transitions_possibles(e):
12     t = []
13     if (e["medecin_libre"] == 1 and e["siege_libre"] == 1
14         and e["salle_attente"]):
15         t.append("Appel du patient prioritaire")
16     if e["consultation"]:
17         t.append("Fin de consultation")
18     return t
19
20 def appliquer_transition(e, transition):
21     n = deepcopy(e)
22     if transition == "Appel du patient prioritaire":
23         p = max(n["salle_attente"], key=lambda x: PRIORITES[x[1]])
24         s = list(n["salle_attente"]); s.remove(p)
25         n["salle_attente"] = tuple(s)
26         n["consultation"] = (p,)
27         n["medecin_libre"] = 0; n["medecin_occupe"] = 1
28         n["siege_libre"] = 0; n["siege_occupe"] = 1
29     elif transition == "Fin de consultation":
30         p = n["consultation"][0]
31         n["consultation"] = ()
32         n["sortis"] = n["sortis"] + (p,)
33         n["medecin_libre"] = 1; n["medecin_occupe"] = 0
34         n["siege_libre"] = 1; n["siege_occupe"] = 0
35     return n
36
37 def verifier_toutes_proprietes(e, n_patients):
38     # Propriété 1 : sûreté
39     securite = len(e["consultation"]) <= 1
40     # Propriété 3 : absence de deadlock
41     terminal = not e["salle_attente"] and not e["consultation"]
42     pas_deadlock = terminal or len(transitions_possibles(e)) > 0
43     # Propriété 4 : respect des priorités
44     if e["consultation"]:
45         prio_c = PRIORITES[e["consultation"][0][1]]
46         prio_ok = all(PRIORITES[p[1]] <= prio_c
47                       for p in e["salle_attente"])
48     else:
49         prio_ok = True
50     # P-invariants structurels
51     I2 = e["medecin_libre"] + e["medecin_occupe"] == 1
52     I3 = e["siege_libre"] + e["siege_occupe"] == 1
53     I4 = (len(e["salle_attente"]) + len(e["consultation"])
54          + len(e["sortis"])) == n_patients
55     return securite, pas_deadlock, prio_ok, I2, I3, I4
56
57 def model_checking(etat_initial):
58     n_patients = (len(etat_initial["salle_attente"])
59                  + len(etat_initial["consultation"])
60                  + len(etat_initial["sortis"]))
61     file = deque([(etat_initial, "Etat initial")])
62     visites = set()
63     etats, violations = [], []
64
65     while file:
66         e, tr = file.popleft()
67         h = rendre_hashable(e)

```

```

68         if h in visites:
69             continue
70         visites.add(h)
71         etats.append((e, tr))
72         res = verifier_toutes_proprietes(e, n_patients)
73         if not all(res):
74             violations.append((e, tr) + res)
75         for t in transitions_possibles(e):
76             file.append((appliquer_transition(e, t), t))
77
78     print(f"Marquages explores : {len(etats)}")
79     print(f"Violations detectees: {len(violations)}")
80     if not violations:
81         print("SUCCEs : toutes les proprietes sont verifiees.")
82     return etats, violations
83
84 def afficher_etats(etats):
85     for i, (etat, transition) in enumerate(etats):
86         print(f"\nM{i} - {transition}")
87         print("Salle attente :", etat["salle_attente"])
88         print("Consultation  :", etat["consultation"])
89         print("Sortis          :", etat["sortis"])
90
91
92 def main():
93     etat_initial = {
94         "medecin_libre": 1,
95         "medecin_occupe": 0,
96         "siege_libre": 1,
97         "siege_occupe": 0,
98         "salle_attente": (
99             ("A", "NON_URGENT"),
100             ("B", "URGENCE_ABSOLUE"),
101             ("C", "URGENT")
102         ),
103         "consultation": (),
104         "sortis": ()
105     }
106
107     etats, violations = model_checking(etat_initial)
108     afficher_etats(etats)
109
110
111 if __name__ == "__main__":
112     main()

```

Listing 3: Model checker exhaustif.

6.3 Complexité et limites

Le BFS garantit la terminaison pour un réseau borné. La complexité est $\mathcal{O}(|M| \cdot |T|)$ où $|M|$ est le nombre de marquages atteignables et $|T|$ le nombre de transitions (ici $|M| = 7$, $|T| = 5$, temps d'exécution négligeable). Pour des modèles industriels, des approches symboliques (BDD dans NuSMV, ordre partiel dans SPIN, heuristiques dans LoLA) seraient nécessaires.

7 Résultats du model checking

7.1 Scénario retenu

Trois patients pré-chargés dans P_2 : Patient A (NON_URGENT), Patient B (URGENCE_ABSOLUE), Patient C (URGENT). Ordre de prise en charge attendu : $B \rightarrow C \rightarrow A$.

7.2 Trace du simulateur

```

1 Patient A arrive (priorité NON_URGENT).
2   Sécurité OK | Ressources OK | Priorités OK | Absence deadlock OK
3 Patient B arrive (priorité URGENCE_ABSOLUE).
4   Sécurité OK | Ressources OK | Priorités OK | Absence deadlock OK
5 Patient C arrive (priorité URGENT).
6   Sécurité OK | Ressources OK | Priorités OK | Absence deadlock OK
7 Patient B appelé en consultation.
8   Sécurité OK | Ressources OK | Priorités OK | Absence deadlock OK
9 Patient B termine et sort.
10 Patient C appelé en consultation.
11  Sécurité OK | Ressources OK | Priorités OK | Absence deadlock OK
12 Patient C termine et sort.
13 Patient A appelé en consultation.
14  Sécurité OK | Ressources OK | Priorités OK | Absence deadlock OK
15 Patient A termine et sort.
16 Vivacité respectée : tous les patients ont été pris en charge.

```

Listing 4: Trace complète de simulation.

7.3 Graphe des marquages atteignables

Le model checker explore 7 marquages $M_0, \dots, M_6$ formant une chaîne linéaire : chaque état admet un unique successeur dans ce scénario.

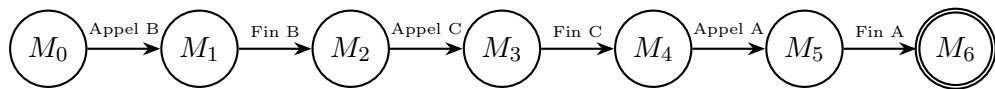


Figure 2: Graphe des marquages atteignables (M_6 : état terminal, double cercle). Les étiquettes des arcs désignent les transitions franchies.

Table 7: Contenu des 7 marquages atteignables.

État	Transition	Salle d'attente	Consultation	Sortis
M_0	État initial	A, B, C	$\emptyset$	$\emptyset$
M_1	Appel prioritaire	A, C	B	$\emptyset$
M_2	Fin consultation	A, C	$\emptyset$	B
M_3	Appel prioritaire	A	C	B
M_4	Fin consultation	A	$\emptyset$	B, C
M_5	Appel prioritaire	$\emptyset$	A	B, C
M_6	Fin consultation	$\emptyset$	$\emptyset$	B, C, A

7.4 Vérification de toutes les propriétés et P-invariants

Table 8: Vérification des 4 propriétés et 3 P-invariants sur chaque marquage.

Marq.	Sûreté	Deadlock	Priorités	$I_2: P_3+P_4=1$	$I_3: P_5+P_6=1$	$I_4: \text{patients}$
M_0	✓	✓	✓	✓	✓	✓
M_1	✓	✓	✓	✓	✓	✓
M_2	✓	✓	✓	✓	✓	✓
M_3	✓	✓	✓	✓	✓	✓
M_4	✓	✓	✓	✓	✓	✓
M_5	✓	✓	✓	✓	✓	✓
M_6	✓	✓	✓	✓	✓	✓
Conclusion : 0 violation détectée sur 7 marquages — toutes les propriétés sont vérifiées.						

7.5 Discussion des résultats

Sûreté. Aucun marquage ne présente deux patients en consultation. Cette propriété est doublement garantie : par le P-invariant y_2 ($P_3 + P_4 = 1$ implique $P_4 \leq 1$, argument structurel indépendant du scénario) et par l’exploration exhaustive (confirmation comportementale).

Vivacité. Les trois patients passent successivement par $P_2 \rightarrow P_7 \rightarrow P_8$. Le marquage final M_6 vérifie $|P_8| = 3 = n$: aucun patient n’est bloqué. La propriété de vivacité est complémentaire au P-invariant y_4 ($P_2 + P_7 + P_8 = n$ constant) : les patients ne sont jamais perdus ou dupliqués.

Absence d’interblocage. M_6 est l’unique état sans successeur ; c’est un état terminal légitime (tous les patients sortis), et non un blocage critique. Tous les marquages intermédiaires admettent au moins un successeur.

Priorités. L’ordre de sortie $B \rightarrow C \rightarrow A$ est scrupuleusement respecté : URGENCE_ABSOLUE avant URGENT avant NON_URGENT. La garde $G(T_4)$ garantit structurellement cette propriété, confirmée colonne par colonne dans le tableau 8.

8 Conclusion et perspectives

8.1 Bilan

Ce projet a mené à bien l’intégralité du cycle de vérification formelle :

- un état de l’art comparé (tableau 2) a justifié le choix des CPN ;
- le système a été instancié en un CPN à 9 places et 5 transitions, avec analyse structurelle complète : matrice d’incidence et 4 P-invariants ;
- quatre propriétés critiques ont été formalisées en LTL/CTL et prouvées logiquement ;
- un simulateur et un model checker Python ont vérifié les 4 propriétés et les 3 P-invariants sur les 7 marquages atteignables, sans violation ;
- le graphe des marquages (figure 2) visualise la séquence complète d’exécution.

8.2 Limites et extensions possibles

- **Multi-ressources** : plusieurs médecins et salles ; concurrence réelle et explosion de l'espace d'états.
- **Contraintes temporelles** : passage aux RdP temporisés (TINA ou UPPAAL) pour vérifier « tout patient en urgence absolue est pris en charge en moins de τ minutes ».
- **Patients dynamiques** : modéliser T_3 avec un nombre maximal de patients simultanés pour conserver la finitude de l'espace d'états.
- **Comparaison avec outil industriel** : le modèle est directement importable dans **CPN Tools** ou **TINA** ; la reproduction de la vérification permettrait de valider notre implémentation *ad hoc* et de tester des propriétés LTL plus complexes.
- **T-invariants** : la séquence $T_4 \cdot T_5$ forme le T-invariant fondamental du modèle fermé ; son calcul formel prouverait structurellement la vivacité.

8.3 Apprentissages

Ce projet a permis de maîtriser concrètement la mécanique du model checking : construction du graphe des marquages par BFS, détection des deadlocks, et traduction des exigences métier en propriétés temporelles vérifiables. La complémentarité entre analyse structurelle (P-invariants, valables pour toute configuration) et exploration exhaustive (confirme sur les scénarios concrets) illustre la richesse des méthodes formelles : deux approches qui s'éclairent mutuellement.

References

- [1] E. M. Clarke, O. Grumberg, D. Peled. *Model Checking*. MIT Press, 1999.
- [2] C. Baier, J.-P. Katoen. *Principles of Model Checking*. MIT Press, 2008.
- [3] A. Pnueli. The Temporal Logic of Programs. *18th Annual Symposium on Foundations of Computer Science (FOCS)*, IEEE, 1977.
- [4] E. M. Clarke, E. A. Emerson. Design and Synthesis of Synchronization Skeletons Using Branching Time Temporal Logic. *Logics of Programs Workshop*, LNCS 131, Springer, 1982.
- [5] J.-P. Queille, J. Sifakis. Specification and Verification of Concurrent Systems in CESAR. *International Symposium on Programming*, LNCS 137, Springer, 1982.
- [6] R. Alur, D. L. Dill. A Theory of Timed Automata. *Theoretical Computer Science*, 126(2), 1994.
- [7] C. A. Petri. *Kommunikation mit Automaten*. Thesis, Universität Hamburg, 1962.
- [8] K. Jensen. *Coloured Petri Nets: Basic Concepts, Analysis Methods and Practical Use*. Springer, 1992.

- [9] C. A. R. Hoare. *Communicating Sequential Processes*. Prentice Hall, 1985.
- [10] R. Milner. *A Calculus of Communicating Systems*. LNCS 92, Springer, 1980.
- [11] G. J. Holzmann. *The SPIN Model Checker: Primer and Reference Manual*. Addison-Wesley, 2003.
- [12] IEC. *IEC 62304: Medical Device Software — Software Life Cycle Processes*. International Electrotechnical Commission, 2006 (amend. 2015).